USER MANUAL · V0.1

flutter-agent

A local Flutter clean-architecture agent. Semantic search over your own Dart corpus, static enforcement of your layer rules, feature scaffolding in your house style, and a lesson store that learns from your Claude sessions.

Runs entirely on this machine. No code leaves your computer. Embeddings are computed on your GPU.

Indexed corpus: 896 files · 1,664 chunks across corereusablepackage, bilhikma, hikmaapp, skygate, attendence.

Two surfaces: MCP tools inside Claude Code, and the `flutter-agent` command line.

Contents

1. **What it is and why it exists** — the problem it solves
2. **How it works** — the pipeline, and what is stored
3. **Installation** — from scratch on a new machine
4. **Registering your projects** — sources and the first index
5. **Using it inside Claude Code** — the MCP tools
6. **Using the command line** — every command, with real output
7. **Scaffolding a feature** — a complete walkthrough
8. **The architecture rules** — R1 to R12, with examples
9. **The knowledge base** — rules, patterns, lessons
10. **The learning loop** — how it gets better over time
11. **Troubleshooting**
12. **Reference** — paths, environment variables, layout

1. What it is and why it exists

Your Flutter conventions live in two places that cannot talk to each other: a prose document, and roughly 900 Dart files that nothing can query.

Every new Claude Code session re-derives your architecture from `CLAUDE.md` — from a description of the rules rather than from the code that already follows them. And every correction you make is lost when that session ends.

flutter-agent closes both gaps:

- **It indexes your real code.** Ask "how do we do pagination" and it answers with *your* `PaginatedResult`, not a generic Flutter tutorial.
- **It enforces your rules statically.** Twelve checks that read the source and decide — no model involved, so they are fast, free and deterministic.
- **It scaffolds in your style.** A generated feature is modelled on `bilhikma/features/archive`, the cleanest instance of the layout in your corpus.
- **It remembers corrections.** When you correct Claude, that correction is recorded, indexed, and recalled in later sessions.

What it is not

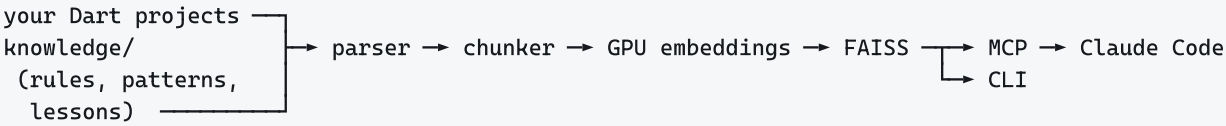
It is not a code generator that replaces thinking, and it does not decide whether code compiles — `flutter analyze` remains the only authority on that, and the agent shells out to it rather than approximating it.

The architecture it encodes

Every feature has exactly two halves, and there is no `domain/`, `entities/`, `use_cases/` or `repo_interface/`:

```
lib/features/<feature>/
  data/
    data_source/   # the ONLY place that talks to a backend or storage
    models/        # fromJson / toJson
    repos/         # delegation, nothing else
  presentation/
    cubit/         # every decision the screen makes
    refactor/      # body sections pulled out of a screen
    screens/       # 30–70 lines: create the cubit, render the body
    widgets/       # small dumb components
```

2. How it works



Step by step

1. **Parse.** A hand-written Dart scanner finds declarations, members, imports and string literals. It is not a Dart compiler and does not try to be — its job is to find boundaries and metadata.
2. **Chunk.** One chunk per top-level declaration. A 600-line Cubit is split at member boundaries, and every part repeats the class header so a chunk from the middle still knows what it belongs to.
3. **Tag.** Each chunk records its project, feature, layer, symbol and line range. That is what makes `--layer data_source` possible.
4. **Embed.** Computed locally on your RTX 4050 with `all-MiniLM-L6-v2` (~90 MB). The full corpus takes about 24 seconds.
5. **Store.** FAISS for the vectors, SQLite for the metadata, keyed so the two can never drift apart.
6. **Serve.** Through MCP to Claude Code, and through the CLI to you.

> **Indexing is incremental.** Only files whose SHA-256 changed are re-embedded. Re-indexing after editing one Cubit costs one embedding call, not nine hundred — typically under two seconds.

Why the lexing matters

Counting braces over raw text breaks the moment a brace appears inside a comment, a string, or a `${...}` interpolation. Everything therefore runs over a *masked* copy of the source in which comments and string literals are blanked out, position for position. This handles nested block comments, raw and triple-quoted strings, and the legal-but-awkward `'${map['key']}'`.

Braces are also only counted outside parentheses, because Dart spells named parameters `({required this.repo})` — counting those as a body brace cuts every such constructor in half.

Validated on your corpus: 885 files, 1,332 declarations, 8,137 members, zero crashes.

3. Installation

Already installed on this machine. This section is for a new machine or a teammate.

Requirements

`Python`	3.10 or newer
`GPU`	Optional. An NVIDIA card makes indexing fast; without one it falls back to CPU automatically.
`Disk`	~5 GB if a new Python environment is built (the CUDA PyTorch wheel is a ~2.8 GB download)
`Flutter`	Only needed if you want `flutter analyze` to verify generated code

Install

```
cd C:\Users\IRD-83\Desktop\local_flutter_agent
.\install.ps1
```

> **By default the installer reuses an existing compatible Python environment** rather than building a second one. On this machine it reuses the `docs-mcp` virtualenv, because it already contains the exact dependency set (torch + CUDA, faiss, sentence-transformers, mcp) and a private copy would cost another ~4.9 GB on a disk that is 91% full.
>
> Pass `-OwnVenv` to build a private environment instead.

Flag	Effect
`-OwnVenv`	Build a private `.venv` instead of reusing one (large download)
`-Cpu`	Install the CPU-only PyTorch wheel (small, slower indexing)
`-SkipRegister`	Do not register the MCP server with Claude Code
`-SkipIndex`	Do not build the index at the end
`-ReuseVenv <path>`	Point at a specific `python.exe` to reuse

Put the command on your PATH

The executable lives in the shared environment, so add that folder once:

```
# this session only
$env:PATH += ";C:\Users\IRD-83\Desktop\local-mcp\.venv\Scripts"

# permanent
[Environment]::SetEnvironmentVariable("PATH",
  $env:PATH + ";C:\Users\IRD-83\Desktop\local-mcp\.venv\Scripts", "User")
```

Register with Claude Code

The installer does this. To do it by hand:

```
claude mcp add --scope user flutter-agent -- `
  C:\Users\IRD-83\Desktop\local-mcp\.venv\Scripts\flutter-agent-server.exe
```

Then **restart Claude Code** and run `/mcp`. You should see `flutter-agent – Connected`.

4. Registering your projects

The agent indexes *source roots*: Flutter apps and shared packages. Register the shared package first, then each app.

```
flutter-agent sources add "C:\Users\IRD-83\Desktop\apps\corereusablepackage"
flutter-agent sources add "C:\Users\IRD-83\Desktop\my apps\bilhikma"
flutter-agent sources add "C:\Users\IRD-83\Desktop\apps\hikmaapp"
flutter-agent sources add "C:\Users\IRD-83\Desktop\my apps\skygate"
flutter-agent sources add "C:\Users\IRD-83\Desktop\apps\attendence"

flutter-agent index
```

The index is **global, not per project** — deliberately. A pattern should be findable wherever you first wrote it, so searching while working on `hikmaapp` still surfaces the right answer from `bilhikma`.

What gets indexed

Every `.dart` file under the project, except generated code (`*.g.dart` , `*.freezed.dart` , `*.gr.dart` , `*.config.dart` , `*.mocks.dart`), tests, and the platform folders (`android/` , `ios/` , `web/` , …). Generated code carries no architectural signal and would swamp retrieval.

Checking what is registered

```
flutter-agent sources list
```

```
attendence            app      files=111   chunks=224   C:\...\apps\attendence
bilhikma              app      files=379   chunks=624   C:\...\my apps\bilhikma
corereusablepackage   package  files=19    chunks=21    C:\...\apps\corereusablepackage
hikmaapp              app      files=205   chunks=397   C:\...\apps\hikmaapp
skygate               app      files=171   chunks=351   C:\...\my apps\skygate
```

Remove one with `flutter-agent sources remove bilhikma` , then re-index to drop it from the store.

> **Per-project tuning** is optional. Drop a `.flutter-agent.json` in a project root to override `include` , `exclude` , `max_chunk_chars` or the embedding model for that project only.

5. Using it inside Claude Code

This is the main way to use it. You do not call the tools yourself — you ask normally, and Claude reaches for them.

What you say	What Claude does
"Add a notifications feature to bilhikma"	`flutter_pattern` → `flutter_component` → `flutter_scaffold`
"Review what I wrote in features/profile"	`flutter_check` on that folder
"This body file is too long, split it"	`flutter_refactor_plan`, then performs the split
"How do we handle errors here?"	`flutter_pattern("error handling")`
"Is there a widget for empty states?"	`flutter_component()`

You can also name a tool explicitly: "*use flutter_search to find how we do pagination*".

The ten tools

Tool	What it answers
`flutter_pattern`	How this codebase does X — the rule, a golden example, and past corrections. **Call before implementing.**
`flutter_search`	Semantic search over the Dart corpus, filterable by layer, feature, project or kind
`flutter_component`	What the shared package exports; with a name, its definition plus real call sites
`flutter_check`	Architecture-rule violations for a file, feature or project
`flutter_refactor_plan`	What to extract from an oversized file, largest method first, plus a precedent
`flutter_scaffold`	A complete feature folder plus localization keys. Dry run by default
`flutter_lesson_add`	Record a correction so it is recalled later
`flutter_lessons`	Recall corrections by topic
`flutter_reindex`	Re-scan sources after a lot of editing
`flutter_status`	Index health, registered sources, device, counts

> **Why this makes Claude cheaper as well as better.** Without the agent, answering "how do we do pagination here" means reading several large Dart files into context. `flutter_pattern` returns a few hundred tokens of exactly the relevant declarations instead.

6. Using the command line

Everything the MCP tools do is available directly. All output below is real.

pattern — before you write

```
flutter-agent pattern "error handling"
```

Returns three things at once: the **rule** that governs the topic, the **golden example** from your corpus, and any **lessons** recorded against it. This is the single most useful command.

component — before you write a widget

```
flutter-agent component
```

```
corereusablepackage exports 18 symbol(s):
  src/theme/app_colors.dart          src/widgets/app_empty_state.dart
  src/theme/app_theme.dart           src/widgets/app_error_state.dart
  src/theme/build_context_ext.dart   src/widgets/app_loading_overlay.dart
  src/widgets/app_button.dart        src/widgets/setting_row.dart
  src/widgets/app_text_field.dart    src/services/toast.dart
  src/widgets/app_card.dart          src/services/app_cache_service.dart
  src/widgets/app_avatar.dart        src/services/screen_security_service.dart
  ...
```

With a name, you get the definition plus every real call site:

```
flutter-agent component AppButton
```

search — find anything

```
flutter-agent search "firestore query with where filters" --layer data_source
flutter-agent search "how a cubit is wired into a screen" --layer screen -n 3
flutter-agent search "pagination" --project bilhikma --paths-only
```

Option	Meaning
`-n, --limit`	Number of results (default 5)
`--layer`	`data_source`, `model`, `repo`, `cubit`, `refactor`, `screen`, `widget`, `core`
`--feature`	Feature folder name, e.g. `archive`
`--project`	Project name, e.g. `bilhikma`
`--kind`	`code`, `rule`, `pattern`, `lesson`
`--paths-only`	Locations without the source text

check — after you write

```
flutter-agent check "C:\Users\IRD-83\Desktop\my apps\bilhikma\lib\features\home"
```

```
package=bilhikma  1 error(s), 2 warning(s)

lib/features/home/presentation/refactor/home_screen_body.dart:194
    [R3 error] refactor body is 194 lines, over the 150-line ceiling
    fix: split the body into presentation/refactor/ or presentation/widgets/
lib/features/home/presentation/screens/home_screen.dart:81
    [R3 warning] screen is 81 lines, over the 70-line target
```

> **Scope matters.** Checking one feature reports only that feature. Checking the whole `lib/` additionally runs the project-wide localization audit (R8) — that is where LangKeys / ar.json / en.json drift shows up. On bilhikma that is 130 undeclared translation keys.

refactor — plan a split

```
flutter-agent refactor "C:\...\features\home\presentation\refactor\home_screen_body.dart"
```

```
lib/features/home/presentation/refactor/home_screen_body.dart
  layer: refactor body   lines: 194 (target 120, ceiling 150)
  rules: 1 error(s), 0 warning(s)
    R3 L194: refactor body is 194 lines, over the 150-line ceiling

  split candidates, largest first:
    _HomeSections.build       L101-192 (92 lines) -> presentation/widgets/
    HomeScreenBody.build      L39-83  (45 lines) -> presentation/refactor/
```

Private classes are routed to `widgets/`, public ones to `refactor/`. It returns a plan, not an edit — the restructuring is a judgement call.

Housekeeping

`flutter-agent index`	Re-scan all sources (incremental; add `--force` to rebuild)
`flutter-agent status`	Index health, device, counts by kind and layer
`flutter-agent doctor`	Confirm the GPU is being used
`flutter-agent sources list`	Registered projects
`flutter-agent export-dataset out.jsonl`	Lessons as training data

7. Scaffolding a feature

Generates eight files plus the localization wiring, modelled on `bilhikma/features/archive`.

Step 1 — dry run

```
flutter-agent scaffold notifications --root "C:\Users\IRD-83\Desktop\my apps\bilhikma"
```

```
files:
  [new] lib/features/notifications/data/data_source/notifications_data_source.dart
  [new] lib/features/notifications/data/models/notifications_model.dart
  [new] lib/features/notifications/data/repos/notifications_repo.dart
  [new] lib/features/notifications/presentation/cubit/notifications_cubit.dart
  [new] lib/features/notifications/presentation/cubit/notifications_state.dart
  [new] lib/features/notifications/presentation/refactor/notifications_body.dart
  [new] lib/features/notifications/presentation/screens/notifications_screen.dart
  [new] lib/features/notifications/presentation/widgets/notifications_tile.dart

edits:
  [update] lib/core/localization/lang_keys.dart   - add 4 constant(s)
  [update] assets/translations/ar.json            - add 4 key(s)
  [update] assets/translations/en.json            - add 4 key(s)

localization keys:
  LangKeys.notificationsTitle      = 'notifications_title'
  LangKeys.notificationsEmpty      = 'notifications_empty'
  LangKeys.notificationsError      = 'notifications_error'
  LangKeys.notificationsLoadFailed = 'notifications_load_failed'
```

The dry run prints the full contents of every file. Nothing is written.

Step 2 — apply

```
flutter-agent scaffold notifications --root "C:\...\bilhikma" --write
```

It refuses to overwrite existing files unless you add `--force`.

Step 3 — verify

```
cd "C:\Users\IRD-83\Desktop\my apps\bilhikma"
flutter analyze lib/features/notifications
```

`No issues found!` — the generated code compiles as written. This is tested: the scaffolder is also asserted to pass the agent's own twelve rules.

Step 4 — the one thing left to do

The data source ships a working stub with the error contract already in place. Replace the body of `fetchItems` with the real call:

```
Future<List<NotificationsModel>> fetchItems() async {
  try {
    // replace this with the real backend call
    const List<Map<String, dynamic>> raw = <Map<String, dynamic>>[];
    return raw.map(NotificationsModel.fromJson).toList(growable: false);
  } on NotificationsException {
    rethrow;
  } catch (error) {
    throw NotificationsException(mapError(error));
  }
}
```

Then register it in the service locator — note `registerFactory` for the cubit:

```
getIt
  ..registerFactory(() => NotificationsCubit(repo: getIt()))
  ..registerLazySingleton(() => NotificationsRepo(dataSource: getIt()))
  ..registerLazySingleton(NotificationsDataSource.new);
```

> **Cubits are always `registerFactory`.** A singleton cubit keeps the previous screen's state and emits after `close()` — the single most common cause of "state from the last time I opened this screen".

Naming

The feature name is normalised, so `LessonDetail`, `lesson-detail` and `lesson detail` all produce `lesson_detail` / `LessonDetail` / `lessonDetail` in the right places.

8. The architecture rules

Twelve static checks. No model is involved in any of them — each is a property of the source that can be decided by reading it.

All pattern matching runs against the masked copy of the file, so a rule can **never** fire on a word inside a comment or a string literal.

ID	Rule	Severity
R1	Folder layout — no `domain/`, `entities/`, `use_cases/`, `repo_interface/`, `repo_impl/`, `presentations/`; canonical folder names	error / warning
R2	No `package:<own_project>/…` self-imports inside `lib/`	error
R3	Files stay within their layer's line budget	error / warning
R4	Screens hold no `_build…` layout methods	error
R5	Presentation never calls Firestore, Supabase, SharedPreferences, Dio, http, Hive or secure storage	error
R6	Cubits never touch `BuildContext`, `Navigator`, `ScaffoldMessenger`, dialogs or bottom sheets	error
R7	UI text goes through `LangKeys` — never `'raw_key'.tr()` or a hardcoded string	error / warning
R8	`LangKeys`, `ar.json` and `en.json` describe the same key set	error / warning
R9	Repos delegate without control flow	warning
R10	No `Expanded` / `Flexible` inside an unconstrained scrollable (heuristic)	warning
R11	Models define at least one of `fromJson` / `toJson`	warning
R12	Features carry the canonical subfolders (grouping folders are skipped)	warning

Line budgets (R3)

Layer	Target	Ceiling
Screen	70	150
Refactor body / widget	120	150
Repo	120	250
Model	200	400
Cubit / data source	400	900

A cubit has a generous ceiling because behaviour is genuinely its job. A screen has a tight one because a screen that grew past it has absorbed work belonging to a cubit or a widget.

R6 in practice — the most important rule

```
// wrong — the cubit reaches into the widget tree
Future<void> save() async {
  await repo.save();
  Navigator.of(context).pop();
  ScaffoldMessenger.of(context).showSnackBar(...);
}

// right — the cubit emits, the screen listens
Future<void> save() async {
  await repo.save();
  if (isClosed) return;
  emit(state.copyWith(status: ProfileStatus.saved));
}
```

A cubit outlives any single widget and may emit after that widget is gone. Holding a context produces "used a BuildContext across an async gap" at best, and a crash at worst.

> **`if (isClosed) return;` after every `await`, before every `emit`.** The user can leave the screen mid-request, and emitting on a closed cubit throws. This appears in every cubit in your corpus — it is not optional style.

R7 in practice

```
Text('saved_items'.tr())   // raw key — nothing stops a typo
Text('Saved items')       // hardcoded, never translated
Text(LangKeys.savedItems.tr())
```

The constant is not decoration: `'saved_itmes'.tr()` compiles, ships, and renders the key itself to the user. `LangKeys.savedItmes` does not compile.

Interpolated and non-linguistic strings are correctly ignored — `Text('$count')`, `Text('–')`, `Text('2x')`.

What it found in your projects

Project	Result
`corereusablepackage`	**0 findings** — the clean reference passes
`bilhikma`	10 errors: 3 self-imports, several files over the ceiling; 130 undeclared translation keys
`skygate`	585 errors: 436 self-imports, 177 raw translation strings, 26 files under `views/` / `utils/`

9. The knowledge base

`knowledge/` is plain Markdown you can read, edit and commit. Run `flutter-agent index` after changing it.

Folder	Kind	Contents
`rules/`	`rule`	Why a constraint exists, with real before/after
`patterns/`	`pattern`	Golden examples lifted from the corpus, with commentary
`lessons/lessons.jsonl`	`lesson`	Append-only corrections captured from sessions

What ships today

Rule cards: layering, localization, errors, file-size.
Pattern cards: cubit-and-state, screen-and-body, data-source-and-model, pagination, dependency-injection.

> **This is the part you grow.** Topics without a card fall back to a weaker match. The fastest way to add one is to ask Claude: "*write a knowledge pattern card for image upload based on how skygate does it*" — it reads the corpus, writes the Markdown, and you re-index.

Writing a good card

- **Show real code** from the corpus, not invented examples.
- **Explain the decision, not the syntax.** "Why `clearError` exists" is useful; "copyWith copies the state" is not.
- **Include the wrong version.** Retrieval is much better at matching a mistake someone is about to make when the mistake is written down.
- **One topic per file**, with `##` headings — the chunker splits on those.

10. The learning loop

This is what makes it an agent that improves rather than a search index that stays still.

When to record a lesson

1. **You correct Claude's Flutter code.** Record what was written, what it should have been, and why.
2. `flutter analyze` **rejects generated code.** Record the error and the fix.
3. `flutter_check` **finds a violation that then gets fixed.** Pass the `--rule` id so the lesson surfaces next to that rule.
4. **A convention is discovered that no rule encodes.** If it recurs, promote it to a `rules/` card.

Recording one

```
flutter-agent lesson "cubit registered as singleton" `
  "getIt..registerFactory(() => ArchiveCubit(repo: getIt()));" `
  --wrong "getIt..registerLazySingleton(() => ArchiveCubit(repo: getIt()));" `
  --why "A singleton cubit keeps the previous screen's state and emits after close()." `
  --tag di --tag cubit --layer cubit --rule R6
```

```
Recorded lesson 9e014a3bcf6a and indexed: cubit registered as singleton
```

In Claude Code you do not type this — you say "*remember that*" after a correction, and Claude calls `flutter_lesson_add`.

Recalling

```
flutter-agent lessons "how do I register a cubit in get_it"
```

Lessons also appear automatically inside `flutter-agent pattern` results, which is the point: the correction surfaces next to the pattern it corrects, before the mistake is repeated.

> **Keep `right` concrete.** A lesson whose correction reads "remember to use the repo" is worthless. One containing the five lines that were actually correct is not.

Lessons as training data

```
flutter-agent export-dataset lessons.jsonl
```

Converts every lesson to an instruction/response triple (`system` / `user` / `assistant`). This is the deferred path to fine-tuning a local model on your style — worth revisiting at a few hundred real samples, not before.

On fine-tuning, honestly

On a 6 GB laptop GPU, QLoRA on long Dart files is slow and the payoff is modest: a fine-tune bakes in style but not correctness, and it will still invent APIs. Retrieval plus the rule engine gets you most of the value at a fraction of the cost. The dataset is the real asset; keep collecting it and decide later.

11. Troubleshooting

Symptom	Cause and fix
Tools missing in Claude Code	Restart Claude Code after registering, then `/mcp`. Verify with `claude mcp list`.
`flutter-agent: command not found`	The folder is not on PATH — see section 3, or call the `.exe` by its full path.
Searches return nothing	`flutter-agent status`. If chunks is 0, no sources are registered or indexed.
Indexing is slow / says `device: cpu`	`flutter-agent doctor`. If CUDA is unavailable, PyTorch was installed as the CPU wheel.
Out of GPU memory	Something else is holding VRAM (ComfyUI is a common culprit — it was using 5.0 of 6.1 GB on this machine). Close it, or set `FLUTTER_AGENT_DEVICE=cpu`.
`health: BROKEN`	FAISS and SQLite disagree. Run `flutter-agent index --force`.
Results are stale	Run `flutter-agent index`. It is incremental and usually takes a second or two.
Check floods with R8 warnings	You pointed it at the project root. Point it at one feature folder instead.
A rule fires wrongly	R10 is explicitly a heuristic and says so. For the rest, this is a bug — the rules are tested with positive and negative cases in `tests/test_rules.py`.

Verifying the whole installation

```
flutter-agent doctor      # GPU in use?
flutter-agent status      # index healthy, counts sensible?
flutter-agent sources list
pytest tests -q           # 53 passed
```

12. Reference

Where things live

`C:\Users\IRD-83\.flutter-agent\index`	FAISS index + SQLite metadata
`C:\Users\IRD-83\.flutter-agent\sources.json`	Registered source roots
`…\local_flutter_agent\knowledge\`	Rule cards, patterns, lessons
`…\local_flutter_agent\src\flutter_agent\`	The package itself
`…\local-mcp\.venv\Scripts\`	The shared Python environment and executables

Environment variables

`FLUTTER_AGENT_HOME`	Where the index lives (default `~/.flutter-agent`)
`FLUTTER_AGENT_KNOWLEDGE`	Knowledge folder (default: the repo's `knowledge/`)
`FLUTTER_AGENT_MODEL`	Embedding model (default `all-MiniLM-L6-v2`)
`FLUTTER_AGENT_DEVICE`	`cuda` or `cpu` (default: auto-detect)

Source layout

`config.py`	Paths, defaults, glob matching
`sources.py`	Registry of indexed Flutter projects
`embedder.py`	sentence-transformers on CUDA, CPU fallback
`store.py`	FAISS + SQLite, metadata columns, filtered search
`dart_parser.py`	Masking lexer and brace scanner
`dart_chunker.py`	Declaration-aware chunking
`layers.py`	Path → (project, feature, layer)
`rules.py`	R1–R12
`scaffold.py`	Feature generator and localization wiring
`lessons.py`	The learning loop and dataset export
`indexer.py`	Incremental, multi-source indexing
`service.py`	Formatted, budgeted tool output
`server.py`	MCP stdio server
`cli.py`	The `flutter-agent` command

Daily loop, condensed

```
# before writing
flutter-agent pattern "<topic>"
flutter-agent component

# while writing
flutter-agent scaffold <feature> --root <project>          # dry run
flutter-agent scaffold <feature> --root <project> --write

# after writing
flutter-agent check <feature folder>
flutter analyze

# when corrected
flutter-agent lesson "<topic>" "<correct code>" --wrong "..." --why "..."

# housekeeping
flutter-agent index
```

---

flutter-agent v0.1 — 5,274 lines of Python, 53 tests. Generated 1 October 2026.